

Принцип code review Python кода

Структура review кода

Выставляется оценка кода, прикладываются файлы статических анализаторов Radon и [Pylint](#). Выделяются грубые ошибки. Описываются антипаттерны. К ошибкам и антипаттернам прикладываются ссылки на GitLab на строчку ошибки.

```
1  Оценка: 4
2  radon.log: повышенная цикломатическая сложность (важно!)
3  pylint.log: замечания pep8
4
5  Грубая ошибка много кода в .orx ресурсах:
6  https://git.sbis.ru/some/url/blob/rc-22.610/src/to/object.orx
7
8  Антипаттерн божественный объект:
9  https://git.sbis.ru/some/url/blob/rc-22.610/src/to/module.py: #L10
```

Числовая оценка

Оценка кода вставляется из алгоритма:

```
1  Итоговая оценка = 5 - штраф Radon - штраф Pylint - 1 за наличие грубых
    ошибок.
```

Например: $5 - 1 - 0.5 - 0.5 = 3$ балла

Штраф Radon

- D+ блоков менее 20% от блоков уровня V+ — 1 балл;
- D+ блоков менее 30% от блоков уровня V+ — 2 балла;
- D+ блоков >30% от блоков уровня V+ — 3 балла.

D+ — блоки D, E, F.

Например, вычисляем количество блоков V+ с помощью команды:

```
1  radon cc -nb -as <путь к ресурсам исходника через пробел>
```

Затем, вычисляем количество блоков D+:

```
1  radon cc -nd -as <путь к ресурсам исходника через пробел>
```

Теперь вычисляем процент блоков D+ = $D+ * 100 / V+$.

Штраф Pylint

- =8 — 0 баллов;
- 5, <8 — 0.5 баллов;
- <5 — 1 балл.

Грубые ошибки

За каждую грубую ошибку вычитается 1 балл, за подфункции — только 0.5 балла.

- Более 5ти строк кода в описании Python-метода в .orx-ресурсе (лучше переходить на указание python-функции как endpoint для BL-метода);
- SQL-запросы по вычитыванию данных из БД в цикле;
- Подфункции не для лямбда выражений.

Подфункции не для лямбда выражений

Это способ обойти подсчет цикломатической сложности кода без решения проблемы.

Подфункция — это функция, объявленная внутри другой функции.

```
1 def outter():
2     def inner():
3         ''' это подфункция '''
4         pass
5     inner();
```

В функции создается подфункция, в которую выносят часть кода и подставляется вызов подфункции. Таким образом формально снижается цикломатическая сложность функции, но проблема остается, т.к. подфункция имеет ту же область видимости, что и основная функция.

Частые антипаттерны

Большие модули/классы (Божественный объект)

Файл и/или класс более 1000 строк кода являются большими. Модуль можно разбить на несколько или превратить в пакет. Класс нуждается в декомпозиции.

Глубокая вложенность кода

При вложенности кода более 5-ти необходимо уменьшить вложенность, т.к. это слишком сильно усложняет понимание кода.

Целиком статичные классы

Если класс состоит целиком из классических и/или статичных методов, то такой класс не нужен. Он заменяется модулем с приватными функциями и публичным API.

В экземплярах целиком статичного класса нет состояния, а значит, это состояние не инкапсулировано между процессами/потоками/вызовами. Это провоцирует ошибки, т.к. зачастую такие классы используют как область для создания статичных глобальных переменных.

Абстрактный класс, где все публичные методы абстрактные

Является обходом предыдущего случая за счет создания пустых абстрактных публичных методов и ничем от него не отличается.

Класс хелпер

Класс, в котором публичные методы не используют состояние экземпляра. Нет причин для существования такого класса, как и в случае с целиком статичным классом. Другими словами, экземпляры такого класса не имеют смысла. Поэтому такой класс тоже преобразуется в модуль с функциями.